

Festival

David hraje na festivalu hru, jejíž hlavní cena je výlet k Laguna Colorada. V této hře se používají mince k nákupu losů, přičemž nákup losu může způsobit zisk dalších mincí. Cílem je získat co nejvíce losů.

Hru začíná s hromádkou A mincí. Existuje N losů očíslovaných od 0 do $N - 1$. Pro nákup losu i musí David zaplatit $P[i]$ mincí (a před jeho nákupem mít alespoň $P[i]$ mincí). Každý los si může koupit maximálně jednou.

Navíc je každému losu i ($0 \leq i < N$) přiřazen **typ** označený $T[i]$, totiž celé číslo **mezi 1 a 4 včetně**. Poté, co si David koupí los i , se zbývající počet mincí, které má, vynásobí číslem $T[i]$. Formálně, pokud má v určitém okamžiku hry X mincí a koupí los i (což vyžaduje $X \geq P[i]$), pak po nákupu bude mít $(X - P[i]) \cdot T[i]$ mincí.

Vaším úkolem je určit, které losy by měl David koupit a v jakém pořadí, aby maximalizoval celkový počet **losů** na konci. Pokud existuje více než jedna posloupnost nákupů, která tohoto maxima dosahuje, můžete vrátit libovolnou z nich.

Implementační detaily

Vaším úkolem je implementovat následující funkci:

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : počet mincí, které má David na začátku.
- P : pole délky N udávající ceny losů.
- T : pole délky N udávající typy losů.
- Tato funkce je zavolána právě jednou pro každý vstup.

Funkce by měla vrátit pole R , které udává Davidovy nákupy takto:

- Délka R je rovna maximálnímu počtu losů, které si může koupit.
- Prvky pole jsou indexy losů, které by si měl koupit, v chronologickém pořadí. To znamená, že si David nejdříve koupí los $R[0]$, poté los $R[1]$ a tak dále.
- Žádný prvek se v R neopakuje.

Pokud nelze koupit žádné losy, R by mělo být prázdné pole.

Omezení

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ pro každé i takové, že $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ pro každé i takové, že $0 \leq i < N$.

Podúlohy

Podúloha	Počet bodů	Další omezení
1	5	$T[i] = 1$ pro každé i takové, že $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ pro každé i takové, že $0 \leq i < N$.
3	12	$T[i] \leq 2$ pro každé i takové, že $0 \leq i < N$.
4	15	$N \leq 70$
5	27	David si může koupit všech N losů (v nějakém pořadí).
6	16	$(A - P[i]) \cdot T[i] < A$ pro každé i takové, že $0 \leq i < N$.
7	18	Žádná další omezení.

Příklad

Příklad 1

Uvažme následující volání:

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

David má na začátku $A = 13$ mincí. Může si koupit 3 losy v pořadí níže:

Koupený los	Cena losu	Typ losu	Počet mincí po nákupu
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

V tomto příkladu si David nemůže koupit více jak 3 losy a výše popsaná posloupnost nákupů je jediný způsob, jak si může koupit 3. Funkce by tedy měla vrátit $[2, 3, 0]$.

Příklad 2

Uvažme následující volání:

```
max_coupons(9, [6, 5], [2, 3])
```

V tomto případě si David může koupit oba losy v libovolném pořadí. Tedy by funkce měla vrátit $[0, 1]$ nebo $[1, 0]$.

Příklad 3

Uvažme následující volání:

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

V tomto příkladu má David jen jednu minci, což nestačí na nákup žádného losu. Tedy by funkce měla vrátit $[]$ (prázdné pole).

Ukázkový grader

Formát vstupu:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formát výstupu:

```
S
R[0] R[1] ... R[S-1]
```

Zde S je délka pole R vráceného funkcí `max_coupons`.